

Erick Molina Gehring

Manipulação de Threads em Linguagem de Programação

Relatório técnico de atividade prática solicitado pelo professor Rodrigo Campiolo na disciplina de Sistemas Operacionais do Bacharelado em Ciência da Computação da Universidade Tecnológica Federal do Paraná.

Universidade Tecnológica Federal do Paraná – UTFPR

Departamento Acadêmico de Computação – DACOM

Bacharelado em Ciência da Computação – BCC

Campo Mourão

Abril / 2026

Sumário

1	Introdução	3
2	Objetivos	3
3	Materiais	3
4	Procedimentos e Resultados	3
	4.1 Análise de Threads	3
	4.2 Programação	7
5	Discussão dos Resultados	7
	5.1 Parte 1	7
	5.2 Parte 2	7
6	Conclusões	8

1 Introdução

Diferente do modelo de processos tradicionais, as threads (ou fluxos de execução) permitem que um único processo realize múltiplas tarefas simultaneamente, compartilhando o mesmo espaço de endereçamento e recursos de memória. Isso reduz drasticamente o custo de criação e comunicação se comparado à criação de novos processos via `fork()`.

Este relatório apresenta as atividades práticas realizadas no Laboratório 3, focadas na manipulação de threads utilizando a biblioteca POSIX Threads (pthreads) em ambiente Linux. O trabalho abrange desde a análise de threads em execução no sistema operacional até a implementação de algoritmos paralelos para busca em vetores e processamento de matrizes de grande escala, visando compreender o impacto do paralelismo no tempo de execução e na utilização dos recursos de hardware.

2 Objetivos

O objetivo desta atividade é aprender a compreender as principais operações usadas em threads, desenvolver aplicações usando threads e explorar programação com Threads POSIX (pthreads).

3 Materiais

- Distribuição Linux.
- Ambiente de desenvolvimento para C/C++.
- Comandos do sistema e bibliotecas de programação.

4 Procedimentos e Resultados

4.1 Análise de Threads

1. Identifique no seu sistema Linux quantas threads estão em execução? Qual o processo com o maior número de threads?

Respostas:

Para identificar o **número total de threads em execução no sistema**, foi utilizado o comando:

```
ps -eo nlwp | tail -n +2 | awk '{sum+=$1} END {print sum}'
```

```
erick@sigmacomputer:~$ ps -eo nlwp | tail -n +2 | awk '{sum+=$1} END {print sum}'
2106
```

Figura 1 – Total de threads em execução

A Figura 1 indica que meu sistema possui **2106 threads em execução** no momento.

A explicação de cada parte do comando é a seguinte:

a) **ps -eo nlwp**

- i. **ps**: mostra informações dos processos em execução no sistema;
- ii. **-e**: seleciona todos os processos;
- iii. **-o nlwp**: define o formato de saída, exibindo apenas o número de threads (NLWP – Number of Lightweight Processes) de cada processo.

b) **| (pipe)**: pega a saída do comando anterior e envia como entrada para o próximo comando;

c) **tail -n +2**

- **tail**: normalmente mostra as últimas linhas;
- **-n +2**: começa a exibir a partir da linha 2, isso é usado pois, A primeira linha é o cabeçalho (NLWP), então esse comando remove o cabeçalho.

d) **| awk '{sum+=\$1} END {print sum}'**

- i. **awk**: usado para processar texto linha por linha;
- ii. **{sum+=\$1}**
 - A. **\$1**: primeira coluna (no caso, o número de threads);
 - B. **sum += \$1**: soma esse valor na variável sum.
- iii. **END {print sum}**
 - A. **END**: executa após processar todas as linhas;
 - B. **print sum**: imprime o total acumulado.

Para **identificar o processo com maior número de threads**, foi utilizado o comando:

```
ps -eo pid,comm,nlwp --sort=-nlwp | head
```

```
erick@sigmacomputer:~$ ps -eo pid,comm,nlwp --sort=-nlwp | head
  PID COMMAND      NLWP
  3742 firefox-esr      174
  2357 java           53
  7242 code          40
  4093 Isolated Web Co 37
  2904 gnome-shell    36
 14216 Isolated Web Co 33
 14380 Isolated Web Co 32
  5484 Isolated Web Co 31
  5611 Isolated Web Co 30
```

Figura 2 – Maior número de threads

A Figura 2 indica que o processo **firefox-esr** (**PID 3742**) possui o maior número de threads, com **174 threads**.

A explicação de cada parte do comando é a seguinte:

a) **ps -eo pid,comm,nlwp**

- i. **ps**: mostra informações dos processos em execução no sistema;
- ii. **-e**: seleciona todos os processos;
- iii. **-o pid,comm,nlwp**: define as colunas que queremos ver, sendo pid, o id do processo, comm, nome do comando/programa e nlwp o número de threads do processo.

b) **--sort=-nlwp**

- i. **--sort**: ordena a saída;
- ii. **-nlwp**: ordena pelo número de threads (decrescente).

c) **| head**: mostra apenas as primeiras linhas (por padrão as 10 primeiras).

2. Qual o número máximo de threads que o seu sistema suporta?

Resposta:

Para verificar o número máximo de threads suportado pelo sistema, foram utilizados os seguintes comandos:

```
cat /proc/sys/kernel/threads-max
```

```
ulimit -u
```

```
erick@sigmacomputer:~$ cat /proc/sys/kernel/threads-max
125649
erick@sigmacomputer:~$ ulimit -u
62824
```

Figura 3 – Máximo de threads suportado pelo sistema e que cada usuário pode criar

A Figura 3 indica que o sistema suporta no máximo **125649** threads simultaneamente e que cada usuário pode criar até **62824** processos/threads.

A explicação de cada parte dos comandos é a seguinte:

a) **cat /proc/sys/kernel/threads-max**

- i. **cat**: usado para exibir o conteúdo de arquivos;
- ii. **/proc/sys/kernel/threads-max**: arquivo do sistema que armazena o número máximo de threads que o kernel Linux pode criar.

b) **ulimit -u**

- i. **ulimit**: define ou exibe limites de recursos do sistema para o usuário;
- ii. **-u**: indica o número máximo de processos (ou threads) que um usuário pode criar.

3. Verifique o tempo de execução do programa da questão 2, parte 2, considerando: 1 thread, 2 threads, 4 threads, 8 threads e 16 threads. Descreva o hardware (processador, memória e número de núcleos, tamanho da matriz usada nos testes e o tempo de execução para cada teste).

Resposta:

- **Hardware:**

- **Processador**: AMD Ryzen 7 5700X 8-Core Processor
- **Núcleos Físicos**: 8
- **Núcleos Lógicos (Threads)**: 16
- **Memória**: 16 GB

- **Matriz usada**: 24000x24000

Tabela 1 – Tempo de execução do programa da questão 2, parte 2

Threads	Tempo
1	24,830s
2	24,759s
4	23,368s
8	22,321s
16	22,118s

4.2 Programação

Os programas feitos para cada um dos exercícios descritos na parte 2 da atividade, como o utilizado para gerar os resultados da Tabela 1 foram feitos e executados com sucesso e serão enviados juntos deste relatório.

5 Discussão dos Resultados

5.1 Parte 1

A análise inicial do sistema revelou um ambiente altamente multithreaded, com o navegador **Firefox-esr** destacando-se como o processo com **maior número de threads (174)**. Isso reflete a arquitetura moderna de navegadores, que utilizam threads separadas para renderização de abas, execução de scripts e gerência de rede para evitar que o travamento de uma aba comprometa toda a aplicação.

Em relação aos testes de desempenho (Tabela 1), observou-se uma **redução no tempo de execução à medida que o número de threads aumentou** para o processamento da **matriz de 24000×24000** . O processador **AMD Ryzen 7 5700X**, que possui 8 núcleos físicos e 16 lógicos, apresentou melhorias constantes até 8 threads.

Ao passar de 8 para 16 threads, o ganho de desempenho foi marginal (de 22,321s para 22,118s). Esse comportamento é esperado, pois as 16 threads lógicas compartilham as unidades de execução dos 8 núcleos físicos. Para tarefas intensivas em CPU (cálculo de médias aritméticas e geométricas), o limite do ganho de performance é ditado principalmente pela quantidade de núcleos físicos disponíveis e pelo overhead de gerência das threads pelo sistema operacional.

5.2 Parte 2

Na implementação da busca em vetor, a transição do modelo de processos para o de threads simplificou a lógica de compartilhamento do vetor, que agora reside em memória

comum a todos os fluxos. Contudo, isso introduziu a necessidade de **Exclusão Mútua (Mutex)**. Ao encontrar o valor alvo, o uso do `pthread_mutex_lock` garantiu que o índice global fosse atualizado sem condições de corrida (race conditions), **um problema que não ocorria no modelo de processos devido ao isolamento de memória**.

Quanto ao processamento da matriz, os resultados da Tabela 1 demonstram a eficiência do paralelismo de dados. O tempo de execução caiu de **24,83s (1 thread) para 22,32s (8 threads)**. É notável que, ao utilizar 16 threads, o ganho foi mínimo. Isso ocorre porque o processador AMD Ryzen 7 5700X possui 8 núcleos físicos. As 16 threads lógicas (Hyper-Threading) não dobram a capacidade de processamento aritmético puro, pois compartilham as mesmas unidades de ponto flutuante (FPU) dos núcleos físicos, evidenciando o limite do hardware frente à paralelização de software.

6 Conclusões

Este laboratório permitiu concluir que o uso de threads é significativamente mais eficiente que a criação de processos para tarefas que demandam intenso compartilhamento de dados. Enquanto no Laboratório 2 a divisão de tarefas exigia que cada processo filho operasse em sua própria cópia do vetor, no Laboratório 3 o uso de memória compartilhada reduziu o overhead de memória e simplificou a comunicação.

Por outro lado, a complexidade da programação multithread reside na sincronização. O uso de mutexes e a atenção aos núcleos físicos do CPU são fundamentais para garantir a corretude e o ganho real de performance. A prática demonstrou que o paralelismo não é uma solução de escala infinita, sendo limitado pela arquitetura do processador e pela eficiência das primitivas de sincronização utilizadas.